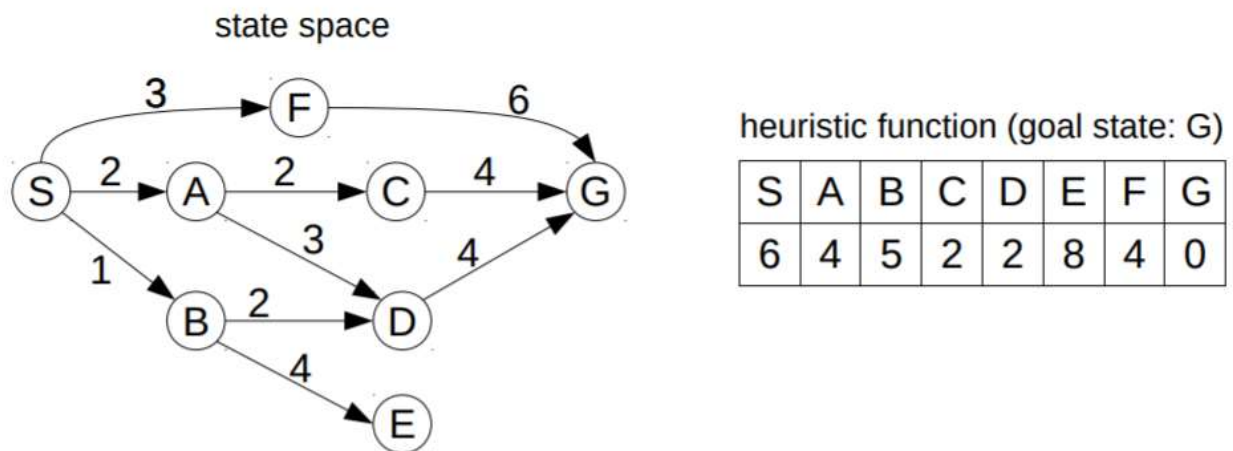


## Esercizio 1 – Ricerca nello Spazio degli Stati

Dato il seguente grafo, applicare l'algoritmo A\*. Utilizzare l'euristica fornita nella tabella.



## Esercizio 2 – Rappresentazione della Conoscenza

**2.a:** Utilizzando una **rete semantica partizionata**, rappresentare la seguente conoscenza: “Secondo Alex, ogni calciatore è alto”

**2.b:** Utilizzando un **grafo concettuale**, rappresentare la seguente conoscenza: “Marco disse che la nazionale di calcio è in forma”.

## Esercizio 3 - Ontologie

Data la seguente ontologia: <http://xmlns.com/foaf/spec/>

- 1) Estendere l'ontologia aggiungendo almeno un paio di object properties e data properties, inserendo delle restrizioni (almeno un paio di Value Constraints e Cardinality Constraints), e popolando l'ontologia con almeno 5 individui.
- 2) Effettuare le seguenti query SPARQL, fornendo una descrizione delle query e uno snapshot dei risultati (ottenuti tramite Protegè):
  - a. Scelta una classe C, una data property P ed un Literal L, ricercare tutte le istanze di C in cui è applicata P e in cui P assume un valore L.
  - b. Ricerca tutte le istanze della classe “Agent”
  - c. Ricercare il numero delle persone presenti nell'ontologia.
  - d. Ricercare tutte le persone il cui numero di telefono inizia con “349”
  - e. Data una Object Property O, ricercare il range ed il dominio

3) Realizzare le seguenti Regole SWRL (fornire anche uno snapshot del risultato della regola eseguita tramite Protegè):

- a. Scegli o crea tre classi differenti, qui indicate in generale come A, B e C, e una object property P (Puoi usare quelle già presenti o che hai creato in precedenza).  
Scrivi una regola per cui se esiste la relazione P tra due individui di A e B, allora l'individuo di A appartiene anche alla classe C.
- b. Scegli o crea tre classi differenti, qui indicate in generale come A, B e C, e due object properties P1 e P2 (Puoi usare quelle già presenti o che hai creato in precedenza).  
Scrivi una regola per cui se esiste una relazione P tra due individui di A e B, allora esiste una relazione P2 tra individui di A e C.
- c. Scegli o crea tre classi differenti, qui indicate in generale come A, B e una data property D1 (Puoi usare quelle già presenti o che hai creato in precedenza).  
Scrivi una regola per cui se un individuo di A possiede la proprietà D, ed essa si trova in un range (a tua scelta), allora esso è individuo anche di B.
- d. Se P è una persona il cui numero di telefono inizia con "349" e il cui sito web è tradotto in lingua italiana e la cui mail termina con ".it" e possiede un documento di identità rilasciato da un comune italiano e conosce almeno una persona di origine italiana, allora P è di nazionalità italiana. Se per eseguire la regola occorrono proprietà non presenti nell'ontologia, si è liberi di inserirle.

### **Esercizio 4 – Logica - Creazione Funtore**

Scrivere un programma in Prolog che ricerchi il percorso più breve tra due nodi "places" all'interno di un grafo. Sono forniti i seguenti fatti, che rappresentano gli archi del grafo:

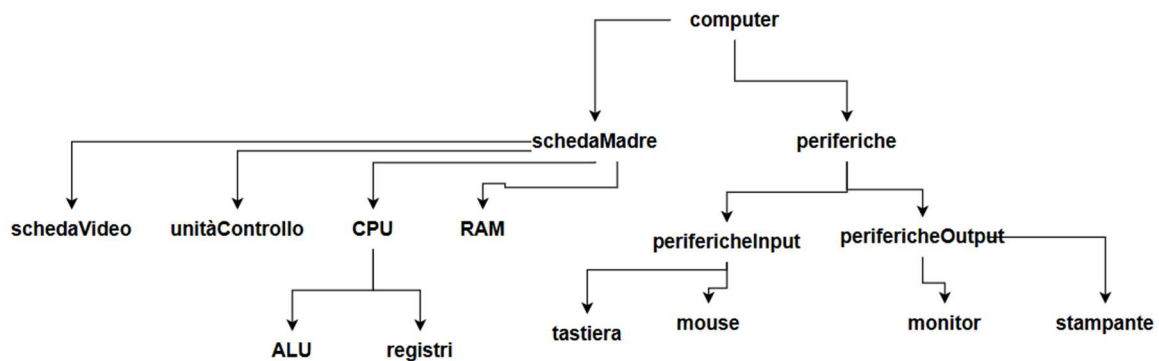
```
edge(placeA, placeB, 11).  
edge(placeA, placeC, 26).  
edge(placeA, placeD, 37).  
edge(placeA, placeE, 13).  
edge(placeA, placeF, 96).  
edge(placeB, placeE, 73).  
edge(placeB, placeC, 14).  
edge(placeC, placeD, 5).  
edge(placeC, placeH, 24).  
edge(placeD, placeH, 71).  
edge(placeE, placeG, 123).  
edge(placeG, placeH, 20).  
edge(placeF, placeH, 29).
```

## Esercizio 5 – Logica

### 5.a: Creazione Funtore

Dati i seguenti fatti, costruire un funtore “computer”:

```
compostoDa(computer, schedaMadre)
compostoDa(computer, periferiche)
compostoDa(schedaMadre, schedaVideo)
compostoDa(schedaMadre, unitàControllo)
compostoDa(schedaMadre, cpu)
compostoDa(schedaMadre, ram)
compostoDa(unitàControllo, alu)
compostoDa(cpu, registri)
compostoDa(periferiche, perifericheInput)
compostoDa(periferiche, perifericheOutput)
compostoDa(perifericheInput, tastiera)
compostoDa(perifericheInput, mouse)
compostoDa(perifericheOutput, monitor)
compostoDa(perifericheOutput, stampante)
```



### 5.b: Ispezione Funtore

Ispezionare ricorsivamente il funtore “computer” creato nel punto precedente, stampando su un file il nome di tutti i componenti.

Comandi per lavorare con file in Prolog:

- **open(path, mode, fd):** per aprire un file in uno specifico **path**. Se il file non esiste viene creato secondo una delle modalità descritte dal secondo parametro “**mode**”: se viene specificata l’opzione “**write**” il vecchio contenuto del file viene rimpiazzato con il nuovo ogni volta che viene effettuata una open, mentre se viene specificata l’opzione “**append**” il nuovo contenuto viene aggiunto in coda al vecchio. Usare **fd** per riferirsi al file.
- **write(fd, testo):** per scrivere sul file.
- **close(fd):** è buona norma chiudere il file una volta terminato l’utilizzo.